

Report: Phishing Detection Using Machine Learning Techniques

Executive Summary

This final project critically reproduces and evaluates the paper "**Phishing Detection Using Machine Learning Techniques**" by Vahid Shahrivari, Mohammad Mahdi Darabi, and Mohammad Izadi for the course **Data Science in Cybersecurity**. The project uses the phishing website dataset published in the authors' GitHub repository and builds a clean, reproducible machine-learning workflow.

The dataset contains **11,055 website samples**, **30 usable feature columns** after removing the artificial index column, and one target column named Result. The original labels are -1 for phishing and 1 for legitimate. For evaluation, phishing is encoded as the positive class.

The best default-threshold model is **Random Forest**, with test Accuracy **0.9765**, Precision **0.9813**, Recall **0.9653**, F1-score **0.9733**, F2-score **0.9685**, MCC **0.9524**, ROC-AUC **0.9965**, and PR-AUC / Average Precision **0.9964**.

The deployment-oriented threshold experiment shows why default benchmark accuracy is insufficient. At the default threshold **0.50**, Random Forest produced **9 false positives** and **17 false negatives**. A cybersecurity-aware threshold of **0.30** reduced false negatives to **8** while keeping false positives at **23**, and improved F2-score from **0.9685** to **0.9777**.

Summary of the Paper

The selected paper studies phishing website detection as a binary classification problem. The authors evaluate several machine-learning methods, including Logistic Regression, Decision Tree, Random Forest, KNN, SVM, AdaBoost, Neural Networks, Gradient Boosting, and XGBoost.

The paper uses engineered website features that describe URL structure, domain information, page behavior, links, and reputation signals. Examples include URL length, IP address usage, SSL state, anchor links, web traffic, page rank, DNS records, and domain age.

The paper reports strong performance for ensemble models, especially Random Forest and XGBoost. It also emphasizes that phishing detection is a cybersecurity problem where false negatives can be especially harmful because malicious sites may reach users.

Critical Evaluation

The paper is valuable because it provides code and data, compares multiple classical machine-learning models, and treats phishing detection as a realistic security classification task.

However, there are important limitations:

- The dataset is based on static engineered features rather than raw modern web traffic.
- Phishing tactics change quickly, so old feature rules may become outdated.

- Strong benchmark accuracy does not prove production readiness.
- The original repository has reproducibility issues, including a misspelled dependency file and unpinned package versions.
- The dataset should ideally be evaluated with time-based splits to test generalization to newer phishing campaigns.

The added PR-AUC, F2-score, and threshold-tuning experiments directly test these concerns. They show that even when accuracy is high, deployment choices such as the classification threshold can significantly change the number of missed phishing sites.

Deployment-Oriented Critical Evaluation

The dataset is static and may not represent modern phishing tactics. Attackers continuously adapt by using HTTPS, realistic domains, URL shorteners, compromised legitimate infrastructure, and brand impersonation strategies that may not be fully captured by older hand-crafted features.

A random train/test split can overestimate performance if similar websites, templates, or feature patterns appear in both training and test data. Real phishing detection requires temporal validation: train on older samples and test on newer attacks.

Accuracy is not sufficient because phishing data may be imbalanced in real traffic. False negatives are dangerous because phishing sites are missed and users may lose credentials or sensitive information. False positives are also costly because legitimate sites may be blocked, causing business disruption and alert fatigue.

Threshold choice should depend on operational security priorities. A model with slightly lower accuracy but higher recall, F2-score, or PR-AUC may be preferable when the security objective is to avoid missed phishing sites. In this project, the selected threshold **0.30** is preferable for a security-focused detector because it reduces false negatives from **17** to **8** while keeping false positives moderate.

Feature Engineering Analysis

The dataset is already numerically encoded. Most features use values such as -1, 0, and 1 to represent suspicious, neutral, or legitimate behavior depending on the feature definition.

The preprocessing steps are:

1. Load data/dataset.csv.
2. Remove the artificial index column.
3. Separate features X and target y.
4. Encode phishing as the positive class: original -1 becomes 1, and original 1 becomes 0.
5. Use random_state=42 for all reproducible operations.
6. Use a stratified 90/10 train/test split.

Scaling is applied to Logistic Regression because optimization is affected by feature magnitude. Tree-based models such as Decision Tree, Random Forest, XGBoost, and Gradient Boosting do not require scaling because they split on feature thresholds rather than using distance calculations.

The strongest target correlations by Spearman correlation are:

Feature	Spearman correlation with original Result
SSLfinal_State	0.7358
URL_of_Anchor	0.7012
web_traffic	0.3651
Prefix_Suffix	0.3486
having_Sub_Domain	0.3047

Spearman correlation is the preferred method because the features are ordinal/categorical encoded values. Pearson correlation assumes continuous numerical variables and linear relationships, which is less suitable here. Kendall correlation is also rank-based, but Spearman is efficient and easier to interpret for this dataset size.

Reproducibility Analysis

The original GitHub repository is useful because it publishes the dataset and implementation notebooks:

https://github.com/fafal-abnir/phishing_detection

Reproducibility issues found:

- The dependency file in the original repository is misspelled as requirment.txt.
- The original README refers to a different misspelled requirements filename.
- Package versions are not pinned.
- The notebooks are exploratory and not organized as a final university project.
- The exact execution environment is not fully documented.

This project improves reproducibility by providing:

- A root-level notebook named final_project_phishing_detection.ipynb.
- A local dataset path: data/dataset.csv.
- Automatic dataset download fallback for Google Colab.
- A clean requirements.txt.
- Fixed random_state=42.

- Stratified train/test split.
- Organized helper functions.
- Saved notebook outputs from a successful full execution.

Experimental Results

The dataset has **11,055 rows** and **32 columns** before preprocessing. After dropping the artificial index column and separating the target, the feature matrix has **11,055 rows** and **30 features**.

Class distribution:

Class	Count	Percentage
Legitimate	6,157	55.69%
Phishing	4,898	44.31%

Train/test split:

Split	Rows	Features
Training set	9,949	30
Test set	1,106	30

Test-set model results at the default threshold:

Model	Accuracy	Precision	Recall	F1	F2	MCC	ROC-AUC	PR-AUC
Random Forest	0.9765	0.9813	0.9653	0.9733	0.9685	0.9524	0.9965	0.9964
Decision Tree	0.9702	0.9790	0.9531	0.9659	0.9581	0.9396	0.9792	0.9693
XGBoost	0.9675	0.9729	0.9531	0.9629	0.9570	0.9341	0.9952	0.9948
Logistic Regression	0.9286	0.9363	0.9000	0.9178	0.9070	0.8552	0.9782	0.9768

Ten-fold cross-validation on the training set:

Model	CV Accuracy Mean	CV F1 Mean	CV F2 Mean	CV ROC-AUC Mean	CV Average Precision Mean
Random Forest	0.9722	0.9684	0.9660	0.9962	0.9957
Decision Tree	0.9638	0.9587	0.9527	0.9745	0.9608
XGBoost	0.9566	0.9504	0.9439	0.9939	0.9927

Model	CV Accuracy Mean	CV F1 Mean	CV F2 Mean	CV ROC-AUC Mean	CV Average Precision Mean
Logistic Regression	0.9276	0.9173	0.9104	0.9788	0.9765

PR-AUC Results

PR-AUC, also called Average Precision, focuses on the precision-recall trade-off for the phishing class. It is useful under class imbalance because it does not reward correct classification of the majority class as strongly as accuracy can.

Model	PR-AUC / Average Precision
Random Forest	0.9964
XGBoost	0.9948
Logistic Regression	0.9768
Decision Tree	0.9693

Random Forest has the highest PR-AUC, meaning it ranks phishing samples very well across possible thresholds.

F2-Score Results

F2-score is suitable for phishing detection because it gives more weight to recall than precision. This is important because a missed phishing site can lead to credential theft or fraud. Precision still matters, but in many security deployments reducing false negatives is the higher priority.

Model	F2-score
Random Forest	0.9685
Decision Tree	0.9581
XGBoost	0.9570
Logistic Regression	0.9070

Random Forest also performs best by F2-score at the default threshold.

Threshold Tuning

The best probabilistic model is Random Forest. The following table evaluates multiple probability thresholds. A lower threshold predicts phishing more aggressively, which usually increases recall and reduces false negatives, but also increases false positives.

Threshold	Precision	Recall	F1	F2	False Positives	False Negatives
0.10	0.8791	0.9939	0.9330	0.9686	67	3
0.20	0.9326	0.9878	0.9594	0.9762	35	6
0.30	0.9545	0.9837	0.9688	0.9777	23	8
0.40	0.9734	0.9714	0.9724	0.9718	13	14
0.50	0.9813	0.9653	0.9733	0.9685	9	17
0.60	0.9853	0.9571	0.9710	0.9626	7	21
0.70	0.9914	0.9429	0.9665	0.9522	4	28
0.80	0.9956	0.9245	0.9587	0.9379	2	37
0.90	1.0000	0.8857	0.9394	0.9064	0	56

The default threshold **0.50** is not necessarily optimal for cybersecurity deployment. It gives strong accuracy and precision, but it misses **17 phishing sites** in the test set. Threshold **0.30** reduces false negatives to **8** and improves F2-score from **0.9685** to **0.9777**, while keeping false positives at **23** out of **616 legitimate test samples**.

Therefore, this project selects **0.30** as the cybersecurity-aware threshold for a warning or analyst-triage setting. For automatic blocking, an organization might choose a higher threshold to reduce false positives.

Evaluation Metrics

The project uses several metrics because accuracy alone is not enough for cybersecurity classification.

- **Accuracy** measures the percentage of all correct predictions.
- **Precision** measures how many websites predicted as phishing are truly phishing.
- **Recall** measures how many true phishing websites are detected.
- **F1-score** is the harmonic mean of precision and recall.
- **F2-score** gives recall more weight than precision.
- **MCC** summarizes all four confusion-matrix cells and is useful when class balance matters.
- **ROC-AUC** measures ranking quality across decision thresholds.
- **PR-AUC / Average Precision** measures precision-recall ranking quality for the phishing class.
- **Confusion Matrix** shows true positives, true negatives, false positives, and false negatives.

In phishing detection, recall is important because missed phishing websites can cause credential theft. Precision is also important because too many false positives can block legitimate websites and cause users to ignore warnings.

Error Analysis

At threshold **0.50**, Random Forest produced **26 total errors**:

Error Type	Count
False Positive: legitimate flagged as phishing	9
False Negative: phishing missed	17

At the selected threshold **0.30**, Random Forest produced:

Error Type	Count
False Positive: legitimate flagged as phishing	23
False Negative: phishing missed	8

A **false positive** means a legitimate website is incorrectly blocked or flagged as phishing. This can harm usability, interrupt business workflows, and create alert fatigue.

A **false negative** means a phishing website is incorrectly classified as legitimate. This is usually more dangerous because a user may visit the malicious website and submit credentials, payment data, or other sensitive information.

The threshold-tuning experiment shows a practical security trade-off. Lowering the threshold from **0.50** to **0.30** increases false positives by **14**, but reduces false negatives by **9**. For many phishing warning systems, that trade-off is acceptable because missed phishing sites are more dangerous than extra warnings.

Conclusions

The reproduction supports the paper's main conclusion that machine-learning methods can perform well on engineered phishing website features. Random Forest achieved the best overall performance in this project.

The deeper deployment-oriented experiments strengthen the critical evaluation. PR-AUC and F2-score show that evaluation should focus on phishing as the positive security event, not only on overall accuracy. Threshold tuning demonstrates that the default **0.50** threshold is not necessarily optimal. A threshold of **0.30** better matches a cybersecurity objective by reducing false negatives while keeping false positives reasonable.

However, the results should still be interpreted carefully. The dataset is static and feature-engineered, so it may not fully represent modern phishing campaigns. A production phishing detector should be evaluated on newer data, time-separated test sets, realistic benign traffic, and operational alert-cost constraints.

Future Work

Future improvements include:

- Evaluate on newer phishing feeds and benign website samples.
- Use time-based train/test splitting to better simulate deployment.
- Add raw URL text features.
- Compare engineered-feature models with deep-learning or transformer-based URL models.
- Add explainability methods such as SHAP.
- Tune detection thresholds for different deployment modes: blocking, warning, or analyst triage.
- Test robustness against adversarial phishing pages designed to bypass static rules.